



Foundry Stablecoin Audit Report

Prepared by: X3 Security

Table of Contents

- ♦ [Table of Contents](#)
- ♦ [Protocol Summary](#)
- ♦ [Disclaimer](#)
- ♦ [Risk Classification](#)
- ♦ [Audit Details](#)
 - [Scope](#)
- ♦ [Executive Summary](#)
 - [Issues found](#)
- ♦ [Findings](#)
- ♦ [High](#)
- ♦ [Medium](#)
- ♦ [Low](#)
- ♦ [Gas](#)

Protocol Summary

This project is designed to be a stablecoin protocol where users can deposit WETH (Wrapped Ether) and WBTC (Wrapped Bitcoin) in exchange for a token pegged to the USD. The system is built to be modular, allowing others to fork the codebase and replace WETH and WBTC with any other assets, while still maintaining the same functionality. The stablecoin protocol includes features like local deployment, testing, and integration with testnets like Sepolia, and is optimized for gas efficiency and security through fuzzing and various test tiers (unit, integration, forked, staging).

Key features include:

- ♦ Easy deployment on testnets (e.g., Sepolia) and mainnet.
- ♦ Integration with test environments using Foundry.
- ♦ Support for testing and fuzzing.
- ♦ A flexible design allowing other users to modify the basket of collateral assets (e.g., swapping WETH/WBTC for other tokens).

Audit Period: July 24th, 2023 - August 5th, 2023

Disclaimer

The X3 Security team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Impact	High	Medium	Low
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

Audit Details

Scope

```
./src/
├── DSCEngine.sol
├── DecentralizedStableCoin.sol
├── libraries
│   └── OracleLib.sol
```

Issues found

Findings Table

Severity	ID	Title
Medium	M-01	[M-01] Risk of incorrect calculation due to negative values in int256 conversion
Medium	M-02	[M-02] Should check return data from Chainlink aggregators
Low	L-01	[L-01] Lack of validation for valid token address in depositCollateral function
Gas	G-01	[G-01] Unchecked arithmetic operations that can't underflow/overflow

Medium

[M-01] Risk of incorrect calculation due to negative values in int256 conversion

Severity: Medium Risk

Likelihood: Medium

Relevant GitHub Links:

[Link to code -](#)

[DSCEngine.sol: L345-](#)

[L347](#)[Link to code -](#)

[DSCEngine.sol: L363-](#)

[L366](#)

Summary

The smart contract `DSCEngine.sol` contains two functions, `getTokenAmountFromUsd` and `getUsdValue`, which fetch token prices from an external Oracle. The vulnerability lies in the incorrect calculation when converting a negative `int256` value to `uint256`, which can lead to large positive numbers instead of the expected negative value.

Vulnerability Details

The functions `getTokenAmountFromUsd` and `getUsdValue` are designed to retrieve token prices in USD from an external Oracle, which returns the price as an `int256` value. The issue occurs when these functions directly convert the price from `int256` to `uint256` without validating if the price is non-negative.

When a negative `int256` is converted to `uint256`, it follows the two's complement representation, resulting in an extremely large positive number. As a consequence, this leads to inaccurate calculations in token

amounts and USD values.

It's important to note that Chainlink Data Feeds return prices as `int256` since some asset prices (e.g., oil futures) can be negative. However, the code provided converts the price directly to `uint256` without ensuring that it is non-negative, leading to potential miscalculations.

Impact

- ♦ **Incorrect Token Amounts:** If a negative price is converted to a large positive number, users may receive incorrect token amounts, disrupting the token's economic balance and leading to potential financial losses.
- ♦ **Misrepresented USD Values:** The contract may calculate incorrect USD values for tokens, causing users to make financial decisions based on incorrect data, further leading to financial imbalances and undermining confidence in the system.

Proof of Concept (PoC)

```
contract IncorrectCalculationPOC {
    int256 negativePrice = -100; // Example negative price

    function convertToInt() public view returns (uint256) {
        uint256 convertedValue = uint256(negativePrice);
        return convertedValue;
        // The result will be equal to
        115792089237316195423570985008687907853269984665640564039457584007913129639836
    }
}
```

Tools Used

- ♦ VS Code

Recommendations

Ensure that the price variable is non-negative before converting it from an `int256` to a `uint256`. This can be addressed by adding a check for negative prices before the conversion.

Here are the updated versions of the relevant functions:

1. `getTokenAmountFromUsd`:

```
function getTokenAmountFromUsd(address token, uint256 usdAmountInWei) public
view returns (uint256) {
    AggregatorV3Interface priceFeed =
    AggregatorV3Interface(s_priceFeeds[token]);
    (, int256 price,,) = priceFeed.staleCheckLatestRoundData();

    // Ensure that the price is non-negative
    require(price >= 0, "Price must be non-negative");

    // Proceed with the calculation
```

```
        return (usdAmountInWei * PRECISION) / (uint256(price) *  
        ADDITIONAL_FEED_PRECISION);  
    }
```

2. **getUsdValue:**

```
function getUsdValue(address token, uint256 amount) public view returns  
(uint256) {  
    AggregatorV3Interface priceFeed =  
    AggregatorV3Interface(s_priceFeeds[token]);  
    (, int256 price,,) = priceFeed.staleCheckLatestRoundData();  
  
    // Ensure that the price is non-negative  
    require(price >= 0, "Price must be non-negative");  
  
    // Proceed with the calculation  
    return ((uint256(price) * ADDITIONAL_FEED_PRECISION) * amount) /  
    PRECISION;  
}
```

[M-02] Should check return data from Chainlink aggregators

Severity: Medium Risk

Likelihood: Medium

Date Modified: Sep 10th, 2024

Relevant GitHub Links:

[Link to code - OracleLib.sol: L21](#)

Summary

The `staleCheckLatestRoundData` function in the `OracleLib.sol` library does not validate the return values obtained from the Chainlink aggregator. It retrieves the latest asset prices via the `latestRoundData` function, but essential checks on parameters such as `roundId`, `updatedAt`, and other critical response values are missing. Without these validation steps, the contract is susceptible to stale or inaccurate price data being used, which can adversely affect liquidity and index price calculations.

Vulnerability Details

The vulnerability stems from the lack of proper checks in the `staleCheckLatestRoundData` function when it fetches data from the Chainlink aggregator. The function retrieves asset prices through the `latestRoundData` function but does not perform checks on the `roundId`, `startedAt`, `updatedAt`, or `answeredInRound` values. This means the function could accept and process outdated or incorrect price information, exposing the system to various risks, such as using stale data for liquidity or pricing calculations.

For example, if the Chainlink oracle is delayed or malfunctioning, it may return stale price data. Without validating the freshness of the returned data, the contract might use this outdated information in its operations, leading to potential imbalances or erroneous financial outcomes.

Impact

- ♦ **Stale Price Data:** If the contract fails to detect stale or incorrect price data, critical financial operations like index price calculations could be performed using outdated values. This could lead to liquidity miscalculations, inaccurate token valuations, and operational inefficiencies.
- ♦ **Unreliable Financial Calculations:** The absence of proper validation checks may result in erroneous pricing data being used for key contract operations, undermining the accuracy of financial decisions within the system.

Proof of Concept (PoC)

The vulnerability can be observed in the `staleCheckLatestRoundData` function:

```
function staleCheckLatestRoundData(AggregatorV3Interface priceFeed)
    public
    view
    returns (uint80, int256, uint256, uint256, uint80)
    {
        (uint80 roundId, int256 answer, uint256 startedAt, uint256 updatedAt,
uint80 answeredInRound) =
            priceFeed.latestRoundData();

        // Missing checks for roundId, answer, startedAt, and answeredInRound

        uint256 secondsSince = block.timestamp - updatedAt;
        if (secondsSince > TIMEOUT) revert OracleLib__StalePrice();

        return (roundId, answer, startedAt, updatedAt, answeredInRound);
    }
```

Tools Used

- ♦ VS Code
- ♦ Manual Review

Recommendations

To address this vulnerability, consider adding missing validation checks for the return data from the Chainlink aggregator. Below is an example of how these checks can be added:

```
(uint80 roundId, int256 answer, uint256 startedAt, uint256 updatedAt, uint80
answeredInRound) =
    priceFeed.latestRoundData();

// Add the following checks
require(roundId > 0, "Invalid roundId"); // Ensure the roundId is valid
require(answeredInRound >= roundId, "Stale price"); // Ensure the price is from a
```



```
valid round
require(answer > 0, "Invalid answer"); // Ensure the price is non-negative and
valid
require(startedAt > 0 && startedAt <= block.timestamp, "Invalid startedAt"); //
Ensure startedAt is a valid timestamp
```

Low

[L-01] Lack of validation for valid token address in `depositCollateral` function

Severity: Low Risk

Likelihood: Medium

Date Modified: Sep 10th, 2024

Relevant GitHub Links:

[Link to code - DSCEngine.sol: L102-L107](#)

[Link to code - DSCEngine.sol: L149-L161](#)

Summary

The `DSCEngine.sol` contract contains a vulnerability in the `depositCollateral` function, where the `isAllowedToken` modifier does not adequately validate whether the provided `tokenCollateralAddress` is a valid ERC20 token contract. The modifier only checks that the price feed for the token exists (i.e., is not `address(0)`), but it does not confirm if the token adheres to the ERC20 standard. This can lead to the acceptance of non-ERC20 tokens, potentially causing errors in contract execution and incorrect handling of token balances.

Vulnerability Details

The `depositCollateral` function relies on the `isAllowedToken` modifier to ensure that the `tokenCollateralAddress` is valid. However, the current implementation only checks if the token's price feed exists, without verifying if the token is a legitimate ERC20 contract. This means that non-ERC20 token contracts could be accepted as collateral, leading to potential errors when the contract tries to interact with them using the `IERC20` interface.

This issue can result in improper token handling, as non-ERC20 tokens do not implement the standard ERC20 functions, such as `transferFrom`. Attempting to interact with non-compliant tokens may cause the contract to revert or behave unpredictably.

Impact

- ♦ **Invalid Token Deposits:** Users may deposit non-ERC20 tokens as collateral, causing issues when the contract tries to handle these invalid tokens. The amounts could be added to the `s_collateralDeposited` array, potentially leading to inconsistencies in the contract's state.

- ♦ **Contract Incompatibility:** When interacting with non-ERC20 tokens, operations such as `transferFrom` (used to transfer collateral from users) will fail, causing contract execution errors and potentially breaking functionality.

Proof of Concept (PoC)

The vulnerable portion of code in `DSCEngine.sol`:

```
modifier isAllowedToken(address token) {  
    if (s_priceFeeds[token] == address(0)) {  
        revert DSCEngine__NotAllowedToken();  
    }  
    _;  
}
```

This modifier only checks if the price feed exists and does not validate the token's compatibility with ERC20 standards.

Tools Used

- ♦ VS Code
- ♦ Manual Review

Recommendations

To address this issue, ensure that the `isAllowedToken` modifier validates whether the token is a legitimate ERC20 token contract. This can be done by utilizing the `IERC20` interface and checking for the existence of standard ERC20 functions such as `totalSupply`. The implementation can use `try/catch` blocks to ensure that only valid ERC20 tokens are accepted.

Recommended updated modifier:

```
modifier isAllowedToken(address token) {  
    // Check if the token is not address(0)  
    if (s_priceFeeds[token] == address(0)) {  
        revert DSCEngine__NotAllowedToken();  
    }  
  
    // Check if the token is a valid ERC20 token contract  
    try IERC20(token).totalSupply() returns (uint256) {  
        // If the call succeeds, it's a valid ERC20 token contract  
        _;  
    } catch {  
        // If the call fails, it's not a valid ERC20 token contract  
        revert("Invalid ERC20 token");  
    }  
}
```

[G-01] Unchecked arithmetic operations that can't underflow/overflow

Severity: Gas Optimization

Likelihood: High

Date Modified: Sep 10th, 2024

Relevant GitHub Links:

[Link to code - DSCEngine.sol: L252](#)

[Link to code - OracleLib.sol: L29](#)

[Link to code - DSCEngine.sol: L118](#)

[Link to code - DSCEngine.sol: L353](#)

Summary

Starting with Solidity version 0.8+, arithmetic operations on unsigned integers include implicit overflow and underflow checks. However, in cases where these checks are unnecessary (e.g., when a comparison has already been made before the operation), gas can be saved by using an **unchecked** block. Using this optimization can save approximately 25 gas per instance.

Vulnerability Details

The project contains several instances where arithmetic operations are performed, and overflow or underflow is impossible. Without using an **unchecked** block, unnecessary gas is consumed. By wrapping these operations in an **unchecked** block, the system could optimize gas usage.

Identified instances:

1. Line 252 (DSCEngine.sol):

```
uint256 totalCollateralToRedeem = tokenAmountFromDebtCovered +  
bonusCollateral;
```

In this instance, both variables have been validated, so there's no need to check for underflow or overflow.

2. Line 118 (DSCEngine.sol):

```
for (uint256 i = 0; i < tokenAddresses.length; i++) {
```

Since **i** is controlled within the loop and doesn't exceed **tokenAddresses.length**, no overflow or underflow can occur.

3. Line 353 (DSCEngine.sol):

```
for (uint256 i = 0; i < s_collateralTokens.length; i++) {
```

Similarly, this loop is bounded, making overflow or underflow impossible.

4. Line 29 (OracleLib.sol):

```
uint256 secondsSince = block.timestamp - updatedAt;
```

The value of `block.timestamp` is always greater than or equal to `updatedAt`, ensuring that underflow cannot occur.

Impact

Using unchecked arithmetic can reduce gas usage by approximately 25 gas per instance. While this might seem negligible for a single instance, repeated operations (especially in loops) can accumulate gas savings, enhancing the overall performance of the smart contract.

Proof of Concept (PoC)

The following example shows how the unchecked block can be implemented for gas optimization:

1. DSCEngine.sol: L252

```
unchecked {
    uint256 totalCollateralToRedeem = tokenAmountFromDebtCovered +
    bonusCollateral;
}
```

2. DSCEngine.sol: L118

```
unchecked {
    for (uint256 i = 0; i < tokenAddresses.length; i++) {
        // loop code
    }
}
```

3. DSCEngine.sol: L353

```
unchecked {
    for (uint256 i = 0; i < s_collateralTokens.length; i++) {
        // loop code
    }
}
```

4. OracleLib.sol: L29

```
unchecked {  
    uint256 secondsSince = block.timestamp - updatedAt;  
}
```

Tools Used

- ♦ VS Code
- ♦ Manual Review

Recommendations

Use **unchecked** blocks around arithmetic operations where overflow or underflow is not possible. This can be applied in the following instances:

1. **DSCEngine.sol: L252** – Uncheck the addition operation.
2. **DSCEngine.sol: L118** – Uncheck the loop iterator increment.
3. **DSCEngine.sol: L353** – Uncheck the loop iterator increment.
4. **OracleLib.sol: L29** – Uncheck the subtraction of timestamps.